

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV

COURSE CODE: DSA0202

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2

Total Marks:30

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I P Sai Bhavyatha(192424333)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Palacharla Sai Bhavyatha

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. Scenario: Design an algorithm to detect faces in an image and recognize them using the Eigenfaces method.

Task:

- A. Input: A grayscale image (size $M \times N$)
- B. Output: Bounding boxes of detected faces and corresponding recognized names
- C. Requirements: Handle multiple faces in the same image and varying lighting conditions

Pseudocode Task:

1. Preprocess the image (grayscale normalization, histogram equalization)
2. Detect faces using a sliding window or Viola-Jones detector
3. Extract features using Eigenfaces
4. Compare with database of known faces using Euclidean distance
5. Output bounding boxes with recognized labels

2. Scenario: Design an algorithm to detect road lanes in a real-time driving video sequence.

Task:

- A. Input: Video frames from a front-facing camera
- B. Output: Lane lines highlighted on each frame
- C. Requirements: Robust to shadows, curves, and partial occlusion

Pseudocode Task:

1. Capture video frame
2. Convert frame to grayscale
3. Apply Gaussian smoothing to reduce noise
4. Perform edge detection (Canny)
5. Apply Hough Transform to detect straight/curved lines
6. Overlay detected lanes on the original frame
7. Repeat for all frames

ANSWERS

1. Face Detection and Recognition Using Eigenfaces

Input: A grayscale image of size $M \times N$ containing one or more faces.

Output: Bounding boxes of detected faces and the corresponding recognized names.

Constraints:

- Multiple faces may be present in the same image.
- Lighting conditions may vary.
- The system must preprocess the image before recognition.
- Known face images and their Eigenface features are stored in a database.
- Recognition is performed using Euclidean distance.

The algorithm first normalizes the grayscale image and improves its contrast using histogram equalization. Faces are then detected using a Viola-Jones detector or sliding-window technique. Each detected face is aligned and resized to a standard size. Eigenface features are extracted using PCA and compared with the stored database using Euclidean distance. If the distance is below a predefined threshold, the corresponding person's name is assigned; otherwise, the face is labeled Unknown.

Pseudocode:

ALGORITHM: Face_Detection_Eigenface_Recognition

INPUT:

Grayscale image I of size $M \times N$

Database D containing known face images and names

Recognition threshold T

OUTPUT:

Bounding boxes and recognized names of all detected faces

BEGIN

1. Read grayscale image I
2. Preprocess image I
 - a. Normalize pixel intensity values
 - b. Apply histogram equalization
 - c. Apply noise reduction if required
3. Detect faces
Faces $\leftarrow$ Viola_Jones_Detector(I)
4. IF Faces is empty THEN
 Display "No faces detected"
 STOP
END IF
5. FOR each detected face F in Faces DO
 - a. Extract bounding box of F
 - b. Align face using facial landmarks
 - c. Resize F to standard dimensions

```

d. Normalize the face image

e. Project F onto the Eigenface space
   Feature ← PCA_Eigenface_Projection(F)

f. MinimumDistance ← INFINITY
   RecognizedName ← "Unknown"

g. FOR each person P in database D DO

   StoredFeature ← Eigenface_Feature(P)

   Distance ← Euclidean_Distance(
       Feature,
       StoredFeature)

   IF Distance < MinimumDistance THEN
       MinimumDistance ← Distance
       RecognizedName ← Name(P)
   END IF

END FOR

h. IF MinimumDistance > T THEN
   RecognizedName ← "Unknown"
END IF

i. Store:
   BoundingBox(F)
   RecognizedName

END FOR

```

6. Display all bounding boxes with recognized names

END

Control Structures Used:

- IF–ELSE: Checks whether faces are detected and whether recognition confidence is sufficient.
- FOR loop: Processes every detected face.
- Nested FOR loop: Compares each detected face with every known person in the database.
- Functions/Modules: Face detection, preprocessing, Eigenface projection, and Euclidean distance calculation are separated into modules.

Handling Special Cases:

- No face detected: Display "No faces detected."
- Multiple faces: Process each detected face independently.
- Unknown person: If minimum Euclidean distance exceeds the threshold, label the face as "Unknown."
- Poor lighting: Histogram equalization and normalization improve recognition.
- Multiple database entries: Select the person having the minimum Euclidean distance.

Complexity:

If there are F detected faces, K known people, and each Eigenface feature contains P dimensions, the approximate comparison complexity is:

$$O(F \times K \times P)$$

Using precomputed Eigenface features and a reduced number of principal components improves efficiency.

Conclusion:

This algorithm provides a complete solution for detecting and recognizing multiple faces while handling varying illumination and unknown faces. The use of preprocessing, Eigenface feature extraction, Euclidean distance, thresholding, loops, and conditional statements makes the algorithm logically complete and suitable for the given requirements.

2. Real-Time Road Lane Detection

Input: A sequence of video frames obtained from a front-facing vehicle camera.

Output: Each video frame with detected lane lines highlighted.

Constraints:

- Processing must be performed continuously for real-time video.
- The algorithm should handle shadows.
- It should support curved roads.
- It should handle partially occluded lane markings.
- Noise should be reduced before edge detection.

Each video frame is captured and converted to grayscale. Gaussian smoothing reduces noise, followed by Canny edge detection to identify strong lane boundaries. A Region of Interest (ROI) is selected to focus processing on the road. Hough Transform is used to detect lane-line segments. For curved lanes, multiple line segments can be combined or a polynomial fitting method can be used. Previous-frame lane information can be used when lane markings are temporarily occluded.

Pseudocode

ALGORITHM : Real_Time_Lane_Detection

INPUT:

Video V from a front-facing camera

OUTPUT:

Video frames with detected lane lines highlighted

BEGIN

1. Open video stream V
2. Initialize PreviousLane $\leftarrow$ NULL
3. WHILE video frames are available DO
 - a. Capture current frame F
 - b. Convert F to grayscale
Gray $\leftarrow$ Convert_To_Grayscale(F)
 - c. Apply Gaussian smoothing
Smooth $\leftarrow$ Gaussian_Blur(Gray)
 - d. Perform Canny edge detection
Edges $\leftarrow$ Canny_Edge_Detection(Smooth)
 - e. Define Region of Interest

ROI $\leftarrow$ Select_Road_Region(Edges)

f. Detect lane lines

Lines $\leftarrow$ Hough_Transform(ROI)

g. IF Lines are detected THEN

i. Separate left and right lane lines

ii. Fit lines or curves to detected points

iii. Smooth lane positions using
PreviousLane information

iv. CurrentLane $\leftarrow$ Fitted_Lane_Lines

ELSE

i. IF PreviousLane is not NULL THEN

CurrentLane $\leftarrow$ PreviousLane

ELSE

CurrentLane $\leftarrow$ NULL

END IF

END IF

h. IF CurrentLane is not NULL THEN

Overlay CurrentLane on original frame F

END IF

i. Display processed frame F

j. PreviousLane $\leftarrow$ CurrentLane

END WHILE

4. Release video stream

END

Control Structures Used:

- WHILE loop: Continuously processes video frames.
- IF-ELSE: Determines whether lane lines were successfully detected.
- Functions: Grayscale conversion, Gaussian filtering, Canny detection, ROI selection, Hough Transform, and lane fitting are implemented as separate modules.
- Previous-frame logic: Maintains lane information during temporary occlusion.

Handling Special Cases:

Shadows:

Adaptive thresholding and edge-based detection reduce the effect of shadows. The Region of Interest prevents irrelevant areas from affecting detection.

Curved Roads:

Instead of assuming that every lane is perfectly straight, detected lane points can be fitted using a polynomial curve.

Partial Occlusion:

If lane markings disappear temporarily, the algorithm can use lane information from previous frames to estimate the current lane position.

No Lane Detected:

If no lane is detected and no previous lane information is available, the algorithm displays the original frame without lane highlighting.

Noise:

Gaussian smoothing is applied before Canny edge detection to reduce unwanted noise.

Complexity:

For each frame, the major computational operations are grayscale conversion, Gaussian filtering, Canny edge detection, and Hough Transform. Processing only the Region of Interest reduces the number of pixels processed and improves real-time performance.

Conclusion:

The proposed algorithm provides a complete real-time lane detection solution using grayscale conversion, Gaussian smoothing, Canny edge detection, ROI selection, Hough Transform, curve fitting, and previous-frame information. It is designed to handle shadows, curves, noise, and partial occlusion while maintaining efficient continuous video processing.